NPC Behavioral System Architecture

Implementation Workflow & Technical Blueprint

Sumit Chandra | Reg: GF202565240
Priyam Kumar Jha (GF202565177)
B.Tech CSE (Game Design and AR/VR) | Shoolini University

1. High-Level Architecture Strategy

This project merges reactive logic, spatial coordination, and generative decision-making. The system transitions from a stealth prototype into multi-agent logic, culminating in an SLM cognitive layer. The core architecture relies on Python, a local SQL database (SQLite) for memory matrix logging, and behavior trees modifying AI utility scores based on player action history.

2. Phase 1: Adaptive Stealth Prototype

Step 0.5: Map and Level Design

- **Plan:** Convert the university campus into a fully navigable 3D environment by combining photogrammetry scanning with manual 3D modeling to optimize the geometry for pathfinding.
- **Resources Needed:** LiDAR-equipped devices or high-resolution cameras for photogrammetry, 3D modeling software (e.g., Blender, Maya) for mesh cleanup, and game engine integration tools to establish collision meshes and spatial bounds.

Step 1.1: Core Mechanics & Player States

- Establish a 2D environment (e.g., Pygame) or lightweight 3D framework.
- Define player states (Walking, Sprinting, Crouching) and assign Noise Emission integers to each.
- Implement the NavMesh and A* pathfinding algorithm for NPC routing.

Step 1.2: Sensory Inputs & Vector Math

- Implement Vision Cones and Hearing Radii.
- Vision detection utilizes vector normalization and the Dot Product to calculate if the player is within the NPC's FOV.
- Line of Sight is determined via Raycasting (Bresenham's Line Algorithm) to account for obstacles.
- Distance Formula: $D = \sqrt{((P_x - N_x)^2 + (P_y - N_y)^2)}$
- Dot Product (Angle Check): $dp = V_x \cdot F_x + V_y \cdot F_y$

Step 1.3: Finite State Machine (FSM)

- Structure the basic NPC brain using four states: Patrol, Investigate, Chase, and Search.
- Implement the logic to seamlessly transition between these states based on the sensory triggers built in Step 1.2.

3. Phase 2: Multi-Agent Spatial Logic (Sports Dynamics)

Step 2.1: Transition to Utility AI

- Upgrade from isolated FSMs to multi-agent Utility AI to support team dynamics.
- NPCs must evaluate multiple tactical options simultaneously (e.g., Flank, Defend, Push).

Step 2.2: Dynamic Positioning

- Assign real-time spatial scores based on variables such as distance to target and teammate placement.
- The NPC will execute the action with the highest aggregate utility score, creating the illusion of synchronized teamwork.

4. Phase 3: The Cognitive Layer (SLM Integration)

Step 3.1: The Telemetry Memory Matrix

- Connect a SQLite database to the game loop.
- Log player behavior metrics dynamically (e.g., aggression levels, preferred routes, patience).
- Compile this telemetry into a numerical "Player Persona".

Step 3.2: Local SLM Orchestration

- Integrate a local sub-1B parameter Small Language Model (e.g., TinyLlama) into the Python backend.
- Inject the "Player Persona" into the SLM prompt as context. Do not use the SLM for frame-by-frame physics; use it to dictate strategic or attitudinal shifts (e.g., "Defensive", "Aggressive") which in turn modify the Utility AI threshold values.

Step 3.3: Tuning & Debug Dashboard

- Build a Streamlit interface running parallel to the game loop.
- Use this dashboard to monitor SQLite updates, the SLM's prompt context, and shifting Utility AI weights in real-time, allowing for precise mathematical tuning of the adaptation system.